

162140222 黄钰轩

2-2 冯·诺依曼结构与哈佛结构各自的特点是什么？

冯·诺依曼结构的处理器使用同一个存储器，即程序和数据共用一个存储器。  
哈佛结构中程序和数据采用独立的总线来访问存储器和数据存储器。

2-4 简述 ARM 体系结构的技术特征。

单周期操作  
采用加载/存储指令结构  
固定的32位长度指令  
使用便于生成优化代码的地址指令格式  
采用多级流水线技术

2-9 CPSR 的哪一位反映了处理器的状态？若  $CPSR = 0x00000090$ ，分析系统的状态。

(1) 低8位 ( $I, F, T, M[4:0]$ )

(2)  $I=1, F=0, T=0, M[4:0]=0b10000$

禁止外部IRQ中断，允许FIQ快速中断，执行ARM指令，CPU处于用户模式。

2-12 一个字的数据  $0x89ABCDEF$ ，存放在  $0x0C100000 \sim 0x0C100003$  区域，分别说明采用小端模式存储和大端模式存储时，上述4个存储单元所存的数据。

|              |    |              |    |
|--------------|----|--------------|----|
| $0x0C100003$ | 89 | $0x0C100003$ | EF |
| $0x0C100002$ | AB | $0x0C100002$ | CD |
| $0x0C100001$ | CD | $0x0C100001$ | AB |
| $0x0C100000$ | EF | $0x0C100000$ | 89 |
| 小端           |    | 大端           |    |

2-15 AMBA 总线由哪两类总线构成？各自的特点是什么？

(1) 系统总线与外围总线。

(2) 系统总线：用于连接高性能系统组件或高带宽组件。

外围总线：支持低性能的外围接口，用于连接系统的周边组件。

2-16 一个 ARM 微控制器系统使用 AMBA 总线结构,说明如果配置 I<sup>2</sup>C 总线接口、SPI 总线接口、UART 接口、GPIO 端口、SRAM、RTC 接口以及 SDRAM 接口应该如何连接到总线上?

*I<sup>2</sup>C、SPI、UART、GPIO、RTC 连接在外围总线上;  
SRAM、SDRAM 连接在系统总线上。*

补: 对于Cortex-M3处理器, 已知一个片上外设中断源被定义为IRQ18, 它的中断类型号是多少? 中断服务程序入口地址为多少?

*中断类型号*

$$ID = IRQ + 16 = 18 + 16 = 34,$$

*中断服务程序入口地址*

$$Iadd0 = (16 + i) \times 4 = (16 + 18) \times 4 = 136 = 0x00000088.$$

3-3 已知 C 标志=1, R0=0, R1=0x80000008, R2=0x00000005, R3=0x0000019F, R4=1, 有关内存数据分布如表 3.24, ARM 工作在小端模式, 说明下列操作完成的功能, 将指出目标操作数的值。

表 3.24 内存数据分布

| 地址         | 数据   | 地址         | 数据   | 地址         | 数据   | 地址         | 数据   |
|------------|------|------------|------|------------|------|------------|------|
| :          | :    | :          | :    | :          | :    | :          | :    |
| 0x80000000 | 0x00 | 0x80000008 | 0x02 | 0x80000010 | 0x00 | 0x80000018 | 0x00 |
| 0x80000001 | 0x70 | 0x80000009 | 0x01 | 0x80000011 | 0x40 | 0x80000019 | 0x40 |
| 0x80000002 | 0x00 | 0x8000000A | 0x11 | 0x80000012 | 0xF2 | 0x8000001A | 0xF2 |
| 0x80000003 | 0x01 | 0x8000000B | 0x47 | 0x80000013 | 0x01 | 0x8000001B | 0x01 |
| 0x80000004 | 0x75 | 0x8000000C | 0x32 | 0x80000014 | 0x00 | 0x8000001C | 0x07 |
| 0x80000005 | 0x39 | 0x8000000D | 0x30 | 0x80000015 | 0x00 | 0x8000001D | 0x0C |
| 0x80000006 | 0x2A | 0x8000000E | 0x30 | 0x80000016 | 0x1F | 0x8000001E | 0x1F |
| 0x80000007 | 0x00 | 0x8000000F | 0x39 | 0x80000017 | 0xFF | 0x8000001F | 0xFF |

- (1) ADD R0, R1, R3, LSL #2
- (2) ANDNES R0, R1, #0x0F
- (3) LDRB R0, [R1, R2, LSR #2]
- (4) ADCHI R1, R3, R2
- (5) EOR R0, R0, R3, ROR R4
- (6) BIC R1, R2, R3 LSL #2
- (7) MLA R0, R1, R2, R3
- (8) LDR R0, [R1, R4, LSL #3]
- (9) BFI R1, R3, #2, #10
- (10) ORN R1, R2, R4

*(1) 将R3左移2位后与R1的和送入R0中*

$$R0 = R1 + R3 \ll 2 = 0x80000008 + 0x67c = 0x80000684$$

3-3 已知 C 标志=1, R0=0, R1=0x80000008, R2=0x00000005, R3=0x0000019F, R4=1, 有关内存数据分布如表 3.24, ARM 工作在小端模式, 说明下列操作完成的功能, 将指出目标操作数的值。

表 3.24 内存数据分布

| 地址         | 数据   | 地址         | 数据   | 地址         | 数据   | 地址         | 数据   |
|------------|------|------------|------|------------|------|------------|------|
| :          | :    | :          | :    | :          | :    | :          | :    |
| 0x80000000 | 0x00 | 0x80000008 | 0x02 | 0x80000010 | 0x00 | 0x80000018 | 0x00 |
| 0x80000001 | 0x70 | 0x80000009 | 0x01 | 0x80000011 | 0x40 | 0x80000019 | 0x40 |
| 0x80000002 | 0x00 | 0x8000000A | 0x11 | 0x80000012 | 0xF2 | 0x8000001A | 0xF2 |
| 0x80000003 | 0x01 | 0x8000000B | 0x47 | 0x80000013 | 0x01 | 0x8000001B | 0x01 |
| 0x80000004 | 0x75 | 0x8000000C | 0x32 | 0x80000014 | 0x00 | 0x8000001C | 0x07 |
| 0x80000005 | 0x39 | 0x8000000D | 0x30 | 0x80000015 | 0x00 | 0x8000001D | 0x0C |
| 0x80000006 | 0x2A | 0x8000000E | 0x30 | 0x80000016 | 0x1F | 0x8000001E | 0x1F |
| 0x80000007 | 0x00 | 0x8000000F | 0x39 | 0x80000017 | 0xFF | 0x8000001F | 0xFF |

- (1) ADD R0, R1, R3, LSL # 2
- (2) ANDNES R0, R1, # 0x0F
- (3) LDRB R0, [R1, R2, LSR # 2]
- (4) ADCHI R1, R3, R2
- (5) EOR R0, R0, R3, ROR R4
- (6) BIC R1, R2, R3 LSL # 2
- (7) MLA R0, R1, R2, R3
- (8) LDR R0, [R1, R4, LSL # 3]
- (9) BFI R1, R3, # 2, # 10
- (10) ORN R1, R2, R4

(2) 当 R1 与 0x0F 不相等时, 将 R1 与 0x0F 按位与后的结果送入 R0 中

$$R0 = R1 \mid 0x0F = 0x80000008$$

(3) 将 R1+R2>>2 所指内存区域中一个字节的数据送入 R0 中

$$R1 + R2 \gg 2 = 0x80000008 + 0x1 = 0x80000009$$

$$R0 = [0x80000009]_{\text{Byte}} = 0x01$$

(4) 当进位标志为 1, 即 C=1, 且条件码为 'HL', 即无符号大于时, 将 R2 与 R3 的和送入 R1

$$R1 = R2 + R3 = 0x000001A4$$

否则不执行这条指令

(5) 将 R3 循环右移 R4 次后与 R0 按位异或的结果送入 R0 中

$$R3 \text{ ROR } R4 = 0x800000CF, R0 = R0 \wedge 0x800000CF = 0x800000CF$$

(6) 将 R3<<2 后的值按位取反, 再与 R2 按位与, 将结果送入 R1 中

$$\sim(R3 \ll 2) = \sim 0x0000067C = 0xFFFF913$$

$$R1 = R2 \& 0xFFFF913 = 1$$

3-3 已知 C 标志=1, R0=0, R1=0x80000008, R2=0x00000005, R3=0x0000019F, R4=1, 有关内存数据分布如表 3.24, ARM 工作在小端模式, 说明下列操作完成的功能, 将指出目标操作数的值。

表 3.24 内存数据分布

| 地址         | 数据   | 地址         | 数据   | 地址         | 数据   | 地址         | 数据   |
|------------|------|------------|------|------------|------|------------|------|
| :          | :    | :          | :    | :          | :    | :          | :    |
| 0x80000000 | 0x00 | 0x80000008 | 0x02 | 0x80000010 | 0x00 | 0x80000018 | 0x00 |
| 0x80000001 | 0x70 | 0x80000009 | 0x01 | 0x80000011 | 0x40 | 0x80000019 | 0x40 |
| 0x80000002 | 0x00 | 0x8000000A | 0x11 | 0x80000012 | 0xF2 | 0x8000001A | 0xF2 |
| 0x80000003 | 0x01 | 0x8000000B | 0x47 | 0x80000013 | 0x01 | 0x8000001B | 0x01 |
| 0x80000004 | 0x75 | 0x8000000C | 0x32 | 0x80000014 | 0x00 | 0x8000001C | 0x07 |
| 0x80000005 | 0x39 | 0x8000000D | 0x30 | 0x80000015 | 0x00 | 0x8000001D | 0x0C |
| 0x80000006 | 0x2A | 0x8000000E | 0x30 | 0x80000016 | 0x1F | 0x8000001E | 0x1F |
| 0x80000007 | 0x00 | 0x8000000F | 0x39 | 0x80000017 | 0xFF | 0x8000001F | 0xFF |

- (1) ADD R0, R1, R3, LSL #2
- (2) ANDNES R0, R1, #0x0F
- (3) LDRB R0, [R1, R2, LSR #2]
- (4) ADCHI R1, R3, R2
- (5) EOR R0, R0, R3, ROR R4
- (6) BIC R1, R2, R3 LSL #2
- (7) MLA R0, R1, R2, R3
- (8) LDR R0, [R1, R4, LSL #3]
- (9) BFI R1, R3, #2, #10
- (10) ORN R1, R2, R4

(7) 将  $R1 \times R2 + R3$  的结果送入 R0 中

$$R1 \times R2 + R3 = 0x80000008 \times 0x5 + 0x195 = 0x2800001C7,$$

结果溢出, 取低 32 位  $R0 = 0x800001C7$

(8) 将  $R1 + R4 \ll 3$  所指内存区域中一个字的数据送入 R0 中

$$R1 + R4 \ll 3 = 0x80000010$$

$$R0 = [0x80000010] = 0x01F24000$$

(9) 将  $R3[11:2]$  中的数据送入  $R1[9:0]$

$$R1 = 0x80000076$$

(10) 将 R4 按位取反后与 R2 按位或, 并将结果送入 R1

$$R1 = R2 \mid (\sim R4) = 0x5 \mid 0xFFFFFFFF = 0xFFFFFFFF$$



3-4 用汇编语言编写实现下列功能的程序段

$R1=a, R2=b, a, b$  为无符号数

(1) if(( $a \neq b$ ) && ( $a - b > 5$ ))  $a = a + b$ ;

else  $a = a - b$ ;

(2) while( $a \neq 0$ )

{  $b = b + b * 8$ ;

$a--$  ; }

(3) 从  $a$  所指向的地址, 拷贝 20 个 32 位数到  $b$  所指向的地址。

```
(1)      CMP R1, R2
          BNE L2

L1      SUB R1, R1, R2
          B L4

L2      SUB R3, R1, R2
          CMP R3, #5
          BHI L3
          JMP L1

L3      ADD R1, R1, R2
L4      B L4
```

```
(3)      MOV R7, #20
LOOP1    LDR R0, [R1], #4
          STR R0, [R2], #4
          SUB R7, R7, #1
          CMP R7, #0
          BNE LOOP1

LOOP2    B LOOP2
```

```
(2)
LOOP1    CMP R1, #0
          BEQ LOOP2
          ADD R2, R2, R2, LSL #3
          SUB R1, R1, #1
          B LOOP1

LOOP2    B.
```

3-8 写一段 ARM 汇编程序:循环累加队列中的所有元素,直到碰到零值位置,结果放在 R4,源程序末尾处声明队列:

```
Array
DCD 0x11
DCD 0x22
DCD 0x33
DCD 0
```

R0 指向队列头,使用命令 LDR R1,[R0],#4 来装载,累加至 R4,循环直到 R1 为 0,用死循环来停止。

```
MOV R4,#0
LDR R0,=Array
LOOP LDR R1,[R0],#4
CMP R1,#0
BEQ EXIT
ADD R4,R4,R1
ADD R0,R0,#4
SUBS R1,R1,#1
BNE LOOP
EXIT B EXIT
```

补: C语言写出实现将32位变量Var的第28, 20, 16, 7位置位, 第30, 19, 8, 6清零, 并让12和0位取反的程序片段。

```
Var |= (1<<28);
Var |= (1<<20);
Var |= (1<<16);
Var |= (1<<7);
Var &= ~(1<<30);
Var &= ~(1<<19);
Var &= ~(1<<8);
Var &= ~(1<<6);
Var ^= (1<<12);
Var ^= (1<<0);
```